

Git Cheat Sheet



Git Cheat Sheet

Git is a powerful version control system that allows developers to track changes, collaborate efficiently, and manage code history. This cheat sheet provides essential Git commands for everyday use.

1. Initialize a Repository

Create an empty Git repository:

```
git init
```

2. Configure Git (First-Time Setup)

Set your name and email (only required once per machine):

```
git config --global user.name "Your Name"  
git config --global user.email "your-email@example.com"
```

To verify your configuration:

```
git config --list
```

3. Staging and Committing Changes

Add files to the staging area:

```
git add .
```

Commit with a suitable message:

```
git commit -m "Your commit message"
```

4. Linking to a Remote Repository

If you have an empty repository on GitHub and want to link your local repository:

```
git remote add origin <repository-url>
```

Verify the remote URL:

```
git remote -v
```

5. Viewing Commit History

View the commit log:

```
git log
```

View a compact commit history:

```
git log --oneline --graph --decorate --all
```

6. Pushing Changes to GitHub

Push commits to GitHub:

```
git push origin main # or the current branch name
```

If the first push command fails, force push:

```
git push --force origin main
```

7. Cloning a Repository

Download a repository to your local machine:

```
git clone <repository-url>
```

Note: You don't need to run `git remote add origin` after cloning, as it is already set.

8. Pulling Updates from GitHub

Update your local repository with the latest changes from GitHub:

```
git pull origin main # or the branch name
```

9. Undoing Changes

If you want to discard all local changes and return to the last committed state:

```
git reset --hard
```

Undo the last commit but keep the changes:

```
git reset --soft HEAD~1
```

10. Branch Management

View all branches:

```
git branch
```

Create and switch to a new branch:

```
git checkout -b <branch-name>
```

Switch to an existing branch:

```
git checkout <branch-name>
```

Rename a branch:

```
git branch -m <new-branch-name>
```

Set up tracking for a remote branch:

```
git fetch origin  
git branch -u origin/main main
```

Change the default branch to main:

```
git branch -M main
```

Delete a local branch:

```
git branch -d <branch-name>
```

Delete a remote branch:

```
git push origin --delete <branch-name>
```

11. Working with Remote Repositories

Check remote repositories:

```
git remote -v
```

Change the remote URL:

```
git remote set-url origin <new-url>
```

Set the remote HEAD to the default branch:

```
git remote set-head origin -a
```

12. Stashing Changes

If you need to switch branches but have uncommitted changes, stash them:

```
git stash
```

View stashed changes:

```
git stash list
```

Apply the latest stash:

```
git stash apply
```

13. Resolving Merge Conflicts

When a conflict occurs, Git will mark the conflicting areas in the affected files. To resolve:

1. Open the conflicting file(s).
2. Manually edit to fix conflicts.
3. Add the resolved file:

```
git add <file-name>
```

Complete the merge:

```
git commit -m "Resolved merge conflict"
```

14. Useful Git Commands

- Check the status of your working directory:

```
git status
```

- View a file's history:

```
git log -- <file-name>
```

- Show differences before committing:

```
git diff
```

- View remote branches:

```
git branch -r
```

- Show who made changes to a file:

```
git blame <file-name>
```

- Remove a file from Git but keep it locally:

```
git rm --cached <file-name>
```

Happy Coding with Git!